

Web API Concepts

What is a Web API

A Web API, or Web Application Programming Interface, is a contract between two pieces of software that allows them to communicate with each other over the internet using the HTTP protocol. One side makes a request, and the other side sends back a response. The application making the request is called the client, and the application responding is called the server. APIs are what make it possible for a weather app on your phone to pull live data from a remote server, or for a payment button on a website to securely talk to a bank system in the background. Without APIs, every application would need to build and manage its own data and services from scratch.

How the Request and Response Cycle Works

Every interaction with a Web API follows the same basic pattern. The client sends an HTTP request to a specific URL called an endpoint. This request includes a method that describes the intended action. GET is used to retrieve data. POST is used to send new data to the server. PUT replaces an existing resource entirely. PATCH updates only specific fields of a resource. DELETE removes a resource. The server receives this request, processes it, and returns an HTTP response. That response contains a status code that tells the client whether the operation succeeded or failed, and usually a body with the actual data in JSON format. Status codes follow a consistent pattern: codes in the 200 range mean success, 400 range means the client made an error, and 500 range means something went wrong on the server.

The Role of JSON in Web APIs

JSON, which stands for JavaScript Object Notation, is the standard format used to exchange data between a client and a server in modern Web APIs. It is lightweight, human readable, and easy for both machines and developers to parse. A JSON response from an API might contain a user object with fields like name, email, and account status. When you call an API and get data back, that data almost always arrives as a JSON string, which your application then parses into a usable data structure. Understanding how to read, parse, and manipulate JSON is a fundamental skill for anyone working with Web APIs.

REST and Why It Matters

REST, which stands for Representational State Transfer, is an architectural style that defines how Web APIs should be structured. It is not a protocol or a standard but a set of principles. When an API follows these principles, it is called a RESTful API. The most important REST principle is statelessness. Every request from a client to a server must contain all the information the server needs to process it. The server does not store any context or session data between requests. This makes RESTful APIs highly scalable because any server instance can handle any incoming request independently. Another key principle is the use of resource-based URLs. Endpoints should represent things, not actions. The URL structure should reflect the resource being accessed, such as using the path for users to retrieve user data or the path for orders with an identifier to access a specific order. The HTTP method itself describes what action to perform on that resource, so there is no need to encode the action into the URL.

Authentication and Security

Most production Web APIs are not open to everyone. They require some form of authentication to verify the identity of the client making a request. The simplest and most common approach is API keys. An API key is a unique string that identifies your application. You include it in the request header, and the server validates it before processing the request. API keys should never be exposed in frontend code or committed to public code repositories because anyone who has your key can impersonate your application. A more advanced and secure authentication approach is OAuth 2.0. OAuth allows users to grant your application limited access to their data on another service without sharing their password. For example, when you log into a third-party app using your Google account, OAuth is handling that process in the background. Understanding both API key authentication and OAuth is essential for working with real-world APIs like those provided by GitHub, Stripe, or Google.

API Versioning and Documentation

As APIs evolve, changes to endpoints or response formats can break applications that depend on them. Versioning solves this by including a version identifier in the URL, such as v1 or v2. This allows the API provider to introduce changes in a new version while keeping the old version available for existing users. Reading API documentation is as important as knowing how to write code. Good documentation describes every endpoint, the expected request format, possible response structures, authentication requirements, and error handling. Learning to navigate documentation efficiently will save significant time and is a skill that improves with practice.